[Lecture Notebook] Decision Structures and Boolean Logic

MGS3001 Python Programming for Business, Spring 2025

Chad (Chungil Chae)

Thu, 13 March 2025

code_03_00_boolean_expressions.py

```
# code_03_00_boolean_expressions.py
x = 1
y = 0
z = 1
print(x > y)
print(y > x)
print(y >= x)
print(x >= y)
print(x != y)
print(x != z)
print(x == y)
print(x == z)
```

code_03_00_decision_structure_01.py

```
# code_03_00_decision_structure_01.py
sales = float(input('What is your sales total?'))

if sales > 50000:
    bonus = 500
    commission_rate = 0.12
    print('you met your sales quota!')
```

8 ## code_03_01_test_average.py

```
# code_03_01_test_average.py
# This program gets three test scores and displays their average.
# It congratulates the user if theaverage is a high score

# considered a high score.
HIGH_SCORE = 95

# Get the three test scores.
test1 = int(input('Enter the score for test 1: ' ))
test2 = int(input('Enter the score for test 2: ' ))
test3 = int(input('Enter the score for test 3: ' ))

# Calculate the average test score.
average = (test1 + test2 + test3) / 3

# Print the average.
print('The average score is', average)

# If the average is a high score,
# congratulate the user.
if average >= HIGH_SCORE:
    print('Congratulations!')
    print('That is a great average!')
```

9 ## code_03_02_auto_repair_payroll.py

```
# code_03_02_auto_repair_payroll.py
# Named constants to represent the base hours and
# the overtime multiplier.
BASE_HOURS = 40 # Base hours per week
OT_MULTIPLIER = 1.5 # Overtime multiplier

# Get the hours worked and the hourly pay rate.
hours = float(input('Enter the number of hours worked: '))
pay_rate = float(input('Enter the hourly pay rate: '))

# Calculate and display the gross pay.
if hours > BASE_HOURS:
# Calculate the gross pay with overtime.
```



```
# First, get the number of overtime hours worked.
    overtime_hours = hours - BASE_HOURS
# Calculate the amount of overtime pay.
    overtime_pay = overtime_hours * pay_rate * OT_MULTIPLIER
# Calculate the gross pay.
    gross_pay = BASE_HOURS * pay_rate + overtime_pay
else:
# Calculate the gross pay without overtime.
    gross_pay = hours * pay_rate

# Display the gross pay.
print('The gross pay is $', format(gross_pay, ',.2f'), sep='')
```

10 **code_03_03_password.py**

```
# code_03_03_password.py
# This program compares two strings.
# Get a password from the user.
password = input('Enter the password: ')

# Determine whether the correct password
# was entered.
if password == 'mgs3001':
    print('Password accepted.')
else:
    print('Sorry, that is the wrong password.')
```

11 **code_03_04_sort_ names.py**

```
# code_03_04_sort_ names.py
# This program compares strings with the < operator.
# Get two names from the user.
name1 = input('Enter a name (last name first): ')
name2 = input('Enter another name (last name first): ')

# Display the names in alphabetical order. print('Here are the names, listed alphabetically.')
if name1 < name2:
    print(name1)
    print(name2)
```



```
else:
    print(name2)
    print(name1)
```

12 **code_03_05_loan_qualifier.py**

```
# code_03_05_loan_qualifier.py
# This program determines whether a bank customer

# qualifies for a loan.
MIN_SALARY = 30000.0 # The minimum annual salary
MIN_YEARS = 2 # The minimum years on the job

# Get the customer's annual salary.
salary = float(input('Enter your annual salary: '))

# Get the number of years on the current job.
years_on_job = int(input('Enter the number of years employed: '))

# Determine whether the customer qualifies.
if salary >= MIN_SALARY:
    if years_on_job >= MIN_YEARS:
        print('You qualify for the loan.')
    else:
        print(
            'You must have been employed',
            'for at least',
            MIN_YEARS,
            'years to qualify.'
        )
else:
    print(
        'You must earn at least $',
        format(MIN_SALARY, ',.2f'),
        ' per year to qualify.',
        sep=''
        )
```


13 ## code_03_06_grader1.py

```
# code_03_06_grader1.py
# This program gets a numeric test score from the user and displays the corresponding letter grad
A_SCORE = 90
B_SCORE = 80
C_SCORE = 70
D_SCORE = 60

# Get a test
score = int(input('Enter your test score: '))
# Determine the grade.
if score >= A_SCORE:
    print('Your grade is A.')
else:
    if score >= B_SCORE:
        print('Your grade is B.')
 #       print('You need', A_SCORE - score, 'more score to get A grade')
    else:
        if score >= C_SCORE:
            print('Your grade is C.')
        else:
            if score >= D_SCORE:
                print('Your grade is D.')
            else:
                print('Your grade is F.')
```

14 ## code_03_06_grader2.py

```
# code_03_06_grader2.py
# This program gets a numeric test score from the
# user and displays the corresponding letter grade.
# Named constants to represent the grade thresholds
A_SCORE = 90
B_SCORE = 80
C_SCORE = 70
D_SCORE = 60

# Get a test
score = int(input('Enter your test score: '))
# Determine the grade.
```



```
if score >= A_SCORE:
    print('Your grade is A.')
elif score >= B_SCORE:
    print('Your grade is B.')
elif score >= C_SCORE:
    print('Your grade is C.')
elif score >= D_SCORE:
    print('Your grade is D.')
else:
    print('Your grade is F.')
```

15 **code_03_07_loan_qualifier2.py**

```
# code_03_07_loan_qualifier2.py
# This program determines whether a bank customer
# qualifies for a loan.

MIN_SALARY = 30000.0 # The minimum annual salary
MIN_YEARS = 2 # The minimum years on the job

# Get the customer's annual salary.
salary = float(input('Enter your annual salary: '))

# Get the number of years on the current job.
years_on_job = int(input('Enter the number of years employed: '))

# Determine whether the customer qualifies.
if salary >= MIN_SALARY and years_on_job >= MIN_YEARS:
    print('You qualify for the loan.')
else:
    print('You do not qualify for this loan.')
```

16 **code_03_08_loan_qualifier3.py**

```
# code_03_08_loan_qualifier3.py
# This program determines whether a bank customer
# qualifies for a loan.

MIN_SALARY = 30000.0 # The minimum annual salary
```



```
MIN_YEARS = 2 # The minimum years on the job

# Get the customer's annual salary.
salary = float(input('Enter your annual salary: '))

# Get the number of years on the current job.
years_on_job = int(input('Enter the number of years employed: '))

# Determine whether the customer qualifies.
if salary >= MIN_SALARY or years_on_job >= MIN_YEARS:
    print('You qualify for the loan.')
else:
    print('You do not qualify for this loan.')
```

17 **code_03_08_loan_qualifier4_steamlit.py**

```
# code_03_08_loan_qualifier4_steamlit.py
import streamlit as st
# This program determines whether a bank customer
# qualifies for a loan.

MIN_SALARY = 30000.0 # The minimum annual salary
MIN_YEARS = 2 # The minimum years on the job

# Get the customer's annual salary.
salary = float(input('Enter your annual salary: '))

# Get the number of years on the current job.
years_on_job = int(input('Enter the number of years employed: '))

# Determine whether the customer qualifies.
if salary >= MIN_SALARY or years_on_job >= MIN_YEARS:
    print('You qualify for the loan.')
else:
    print('You do not qualify for this loan.')



# Main
# ------------------------------------------------------------
# Create a title for your app
st.title("My first steamlit app")
```



```
"""
Load qualifier
"""


# ------------------------------------------------------------
```

18 **code_03_09_hit_the_target.py**

```
# code_03_09_hit_the_target.py
# Hit the Target Game
import turtle

# Named constants
SCREEN_WIDTH = 600 # Screen width
SCREEN_HEIGHT = 600 # Screen height
TARGET_LLEFT_X = 100 # Target's lower-left X
TARGET_LLEFT_Y = 250 # Target's lower-left Y
TARGET_WIDTH = 25 # Width of the target
FORCE_FACTOR = 30 # Arbitrary force factor
PROJECTILE_SPEED = 1 # Projectile's animation speed
NORTH = 90 # Angle of north direction
SOUTH = 270 # Angle of south direction
EAST= 0 # Angle of east direction
WEST = 180 # Angle of west direction

# Setup the window.
turtle.setup(SCREEN_WIDTH, SCREEN_HEIGHT)

# Draw the target.
turtle.hideturtle()
turtle.speed(0)
turtle.penup()
turtle.goto(TARGET_LLEFT_X, TARGET_LLEFT_Y)
turtle.pendown()
turtle.setheading(EAST)
turtle.forward(TARGET_WIDTH)
turtle.setheading(NORTH)
turtle.forward(TARGET_WIDTH)
turtle.setheading(WEST)
turtle.forward(TARGET_WIDTH)
turtle.setheading(SOUTH)
```



```
turtle.forward(TARGET_WIDTH)
turtle.penup()

# Center the turtle.
turtle.goto(0, 0)
turtle.setheading(EAST)
turtle.showturtle()
turtle.speed(PROJECTILE_SPEED)

# Get the angle and force from the user.
angle = float(input("Enter the projectile's angle: "))
force = float(input("Enter the launch force (1-10): "))

# Calculate the distance.
distance = force * FORCE_FACTOR

# Set the heading.
turtle.setheading(angle)

# Launch the projectile.
turtle.pendown()
turtle.forward(distance)

# Did it hit the target?
if(turtle.xcor() >= TARGET_LLEFT_X and
   turtle.xcor() <= (TARGET_LLEFT_X + TARGET_WIDTH) and
   turtle.ycor() >= TARGET_LLEFT_Y and
   turtle.ycor() <= (TARGET_LLEFT_Y + TARGET_WIDTH)):
   print('Target hit!')
else:
    print('You missed the target.')

turtle.done()
```

